



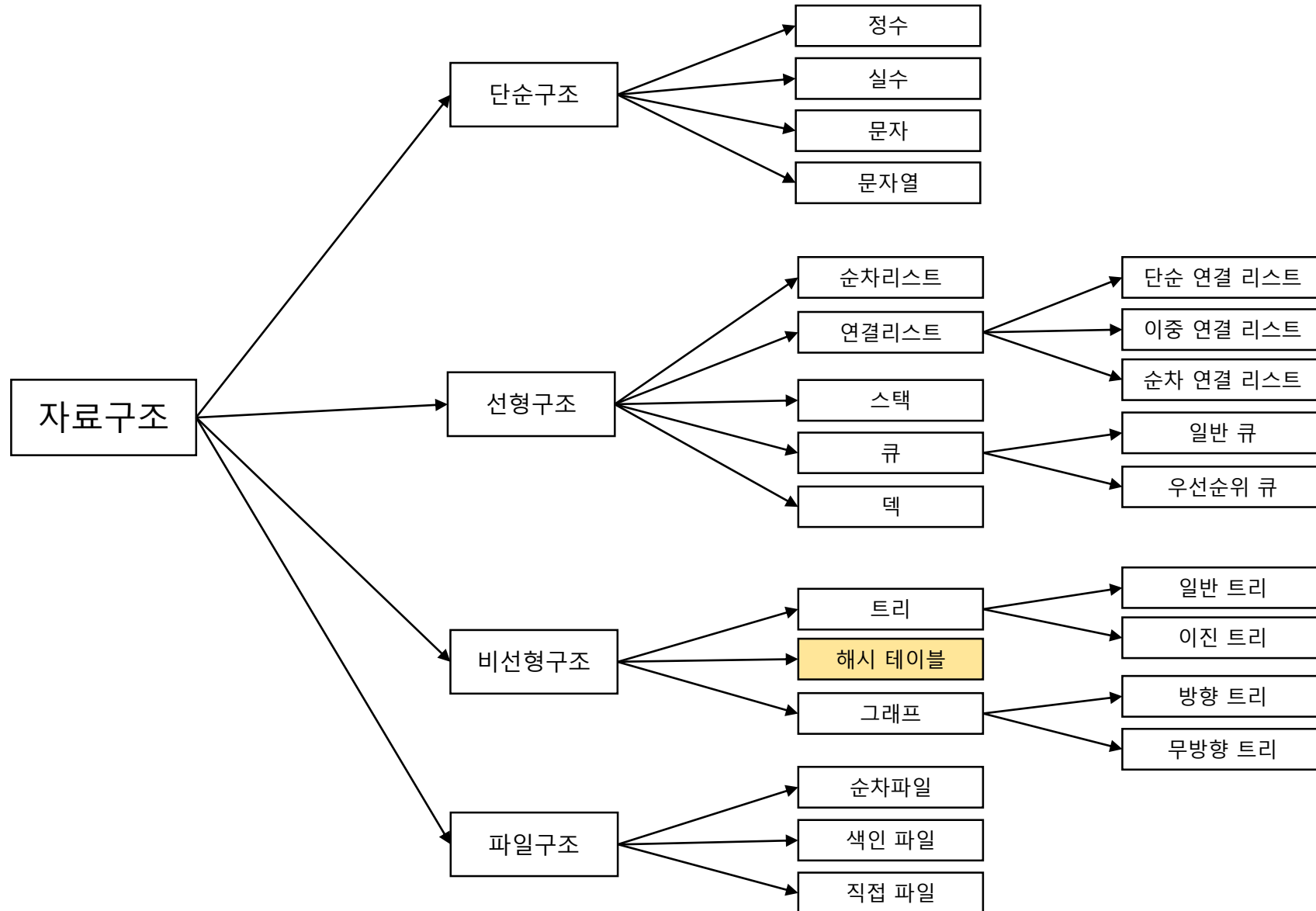
국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

친환경스마트조선기자재학과

데이터처리특론



Data structure



Hash table



hash

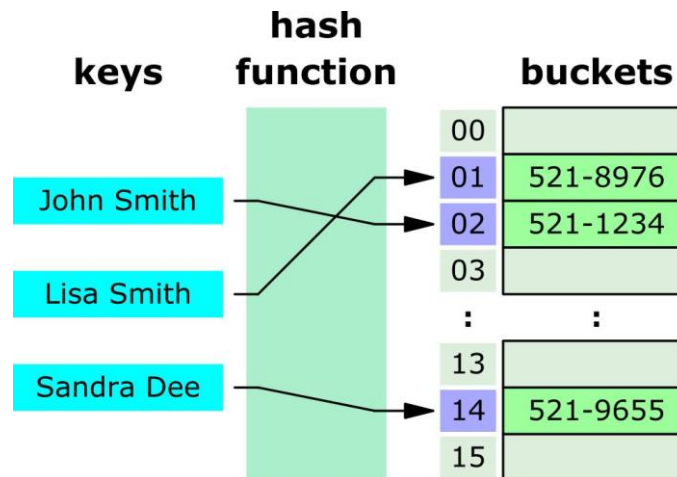
- Hash Brown이 감자를 잘게 다져서 다시 만든 것인 것처럼 Hash Table도, Table을 잘게 다져서 다시 만든 것이라 생각할 수 있다.
- Sequential Search는 성능이 안좋다. $O(N)$ 이기 때문에. 정렬된 Binary Tree의 경우 $O(\log N)$ 의 성능으로 좋다.
- 그런데 배열의 경우는 Index만 알면 $O(1)$ 로 접근이 가능하다.
- 이러한 원리로, 데이터를 이용해 Index 값을 알 수 있으면 굉장히 빠르게 탐색을 할 수 있는 것이다.
- 이것이 Hash Table의 포인트다.

hash

1. 명사 해시(고기와 감자를 잘게 다져 섞어 요리하여 따뜻하게 차려 낸 것)
2. 명사 (=hashish)
3. 명사 (특히 전화기의) 우물 정자(#)

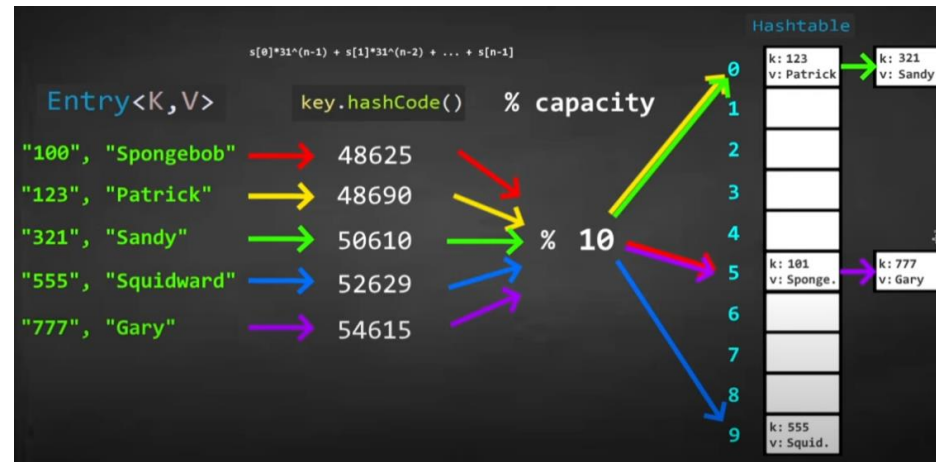
hash

- 해시란 단방향 암호화 기법으로 해시 함수를 이용하여 고정된 길이의 암호화된 문자열로 바뀌버리는 것을 의미한다.
- 자료구조에서의 해싱(Hashing)은, 각각의 데이터를 고유한 숫자 값으로 표현하고(임의의 값을 고정된 길이로 변환), 이를 이용하여 특정 데이터의 존재 여부를 확인하거나 데이터를 추출하는 작업을 의미한다.
- 해시 함수를 이용해서 key를 hash value로 매핑하고, 이 hash value를 index로 삼아 데이터의 value를 buckets(혹은 slots)에 저장했다.



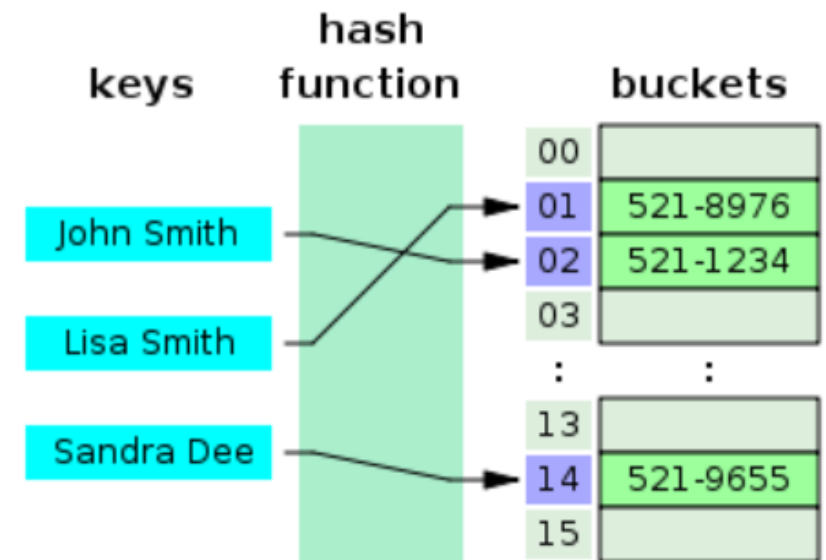
HashTable

- 해시 테이블은 (Key, Value)로 데이터를 저장하는 자료구조 중 하나로 빠르게 데이터를 검색할 수 있는 자료구조이다.
- 해시 테이블이 빠른 검색 속도를 제공하는 이유는 내부적으로 배열(버킷)을 사용하여 데이터를 저장하기 때문이다.
- 해시 테이블은 각각의 Key값에 해시함수를 적용해 배열의 고유한 index를 생성하고, 이 index를 활용해 값을 저장하거나 검색하게 된다.
- 여기서 실제 값이 저장되는 장소를 버킷 또는 슬롯이라고 한다.



HashTable

- 예를 들어 우리가 (Key, Value)가 ("John Smith", "521-1234")인 데이터를 크기가 16인 해시 테이블에 저장한다고 하자.
- 그러면 먼저 $\text{index} = \text{hash_function}(\text{"John Smith"}) \% 16$ 연산을 통해 index 값을 계산한다.
- 그리고 $\text{array}[\text{index}] = \text{"521-1234"}$ 로 전화번호를 저장하게 된다.
- 이러한 해싱 구조로 데이터를 저장하면 Key값으로 데이터를 찾을 때 해시 함수를 1번만 수행하면 되므로 매우 빠르게 데이터를 저장/삭제/조회할 수 있다.
- 해시테이블의 평균 시간복잡도는 $O(1)$ 이다.



HashTable

- 해시 함수에 의해 얻어지는 값을 해시라고 부른다.
- 해시 함수에 key를 줄 경우, key에 해당하는 숫자 index주소를 반환하고, 그 숫자 index에 value를 저장하는 원리로 해시 테이블을 생성한다.
- 해시 테이블은 키(key)와 값(value)으로 데이터를 저장하는 비선형 자료 구조로, 각 키(key)에 대해 고유한 해시 값(hash value)을 생성하고, 이 해시 값을 배열의 인덱스로 사용하여 값을 저장하고 검색하는 방식이다.
- 해시가 매우 빠른 데이터 검색을 할 수 있는 이유는 데이터를 검색할 때 사용할 key와 실제 데이터의 값(value)이 한 쌍으로 존재하고, key값이 배열의 인덱스로 변환되기 때문에 검색과 저장의 평균적인 시간 복잡도가 $O(1)$ 에 수렴하기 때문이다.
- 즉 일반 배열을 순회할 때는 $O(N)$, 선형 시간으로 순회할 아이템이 많을 수록 찾는 시간도 오래걸려 시간복잡도가 증가하지만, 해시 테이블의 경우 $O(1)$ 상수 시간이기 때문에 어떤 요소를 찾는 데 소요되는 건 딱 1개의 스텝이라는 것이다.
- 매우 빠른 데이터 검색을 위한 컴퓨터 소프트웨어 에서 사용된다.

HashTable

- Hash table은 효율적인 탐색(빠른 탐색)을 위한 자료구조로써 key-value 쌍의 데이터를 입력받는다.
hash function h 에 key 값을 입력으로 넣어 얻는 해시값 $h(k)$ 를 위치로 지정하여 key-value Hash table 데이터 쌍을 저장한다.
- Direct-address Table
 - ✓ key 값이 k 인 데이터를 index k 위치에 저장하는 방식

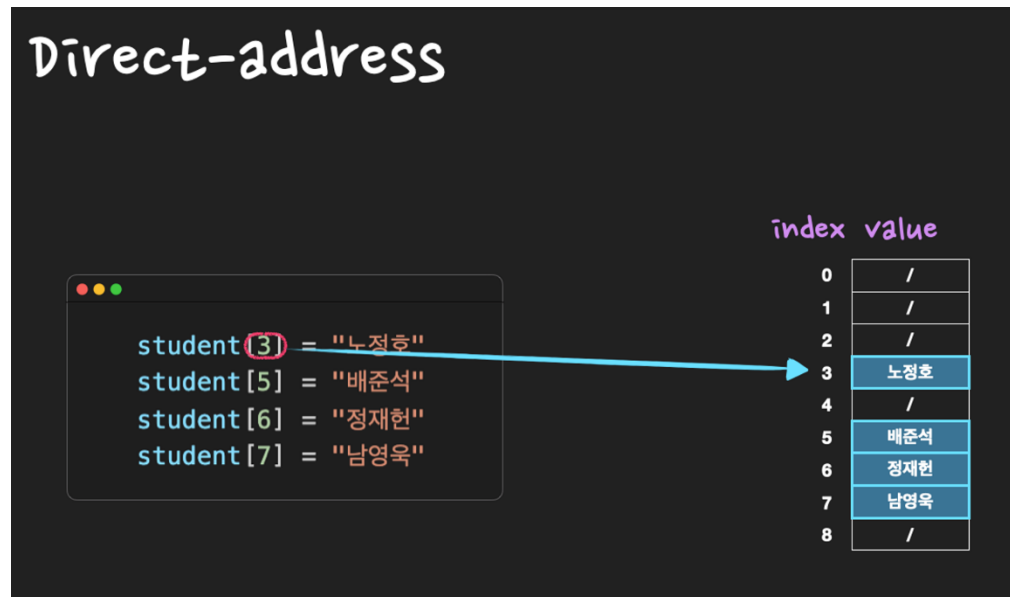
key: 출석번호, value: 이름

(3, 노정호)

(5, 배준석)

(6, 정재현)

(7, 남영욱)



HashTable

- 직접 주소화 방법을 통해 key-value 쌍의 데이터를 저장하고자 하면 많은 문제가 발생

➤ 문제점 1) 불필요한 공간 낭비

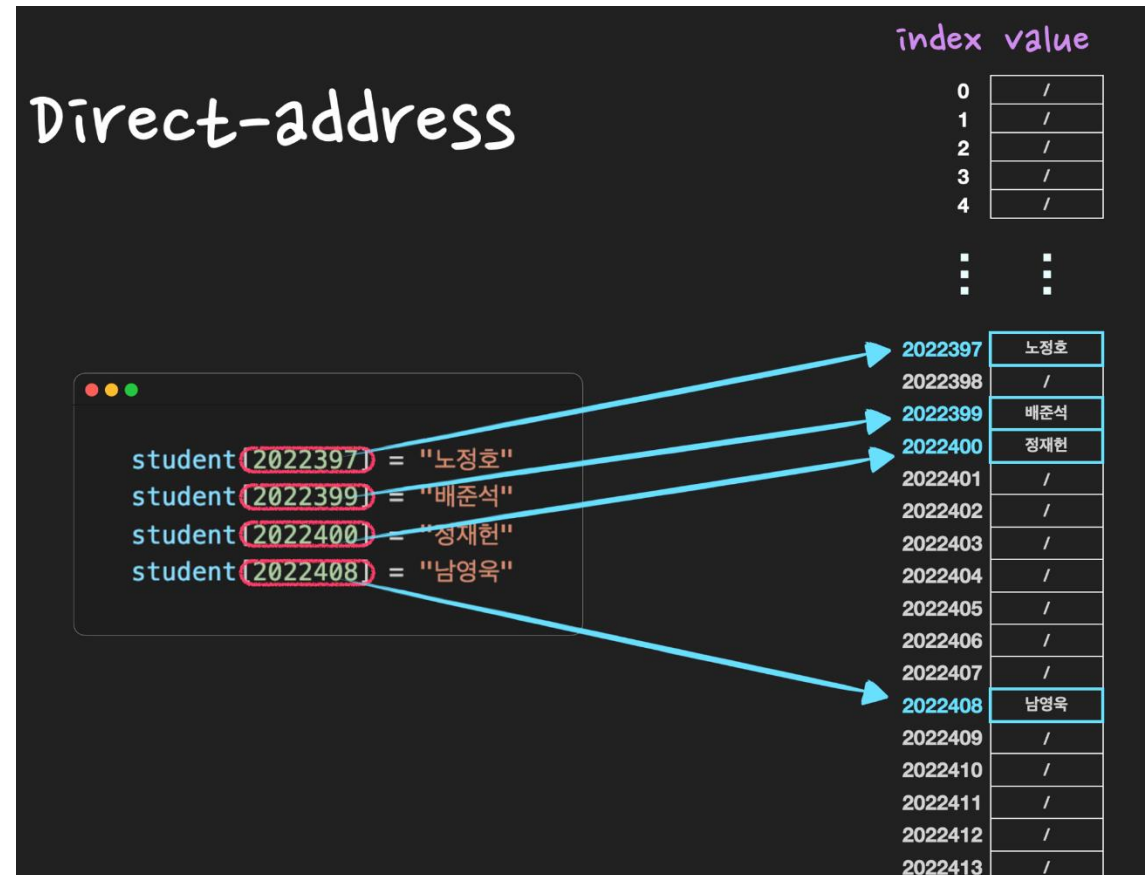
key: 학번, value: 이름

(2022397, 노정호)

(2022399, 배준석)

(2022400, 정재현)

(2022408, 남영욱)



HashTable

- 직접 주소화 방법을 통해 key-value 쌍의 데이터를 저장하고자 하면 많은 문제가 발생
 - 문제점 2) key 값으로 문자열이 올 수 없다.

key: ID, value: 이름

(shwjdgh34, 노정호)

(js9876, 배준석)

(zebra001, 정재현)

(nam1234, 남영욱)

Direct-address

```

student["shwjdgh34"] = "노정호"
student["js9876"] = "배준석"
student["zebra001"] = "정재현"
student["nam1234"] = "남영욱"
    
```

?

index value

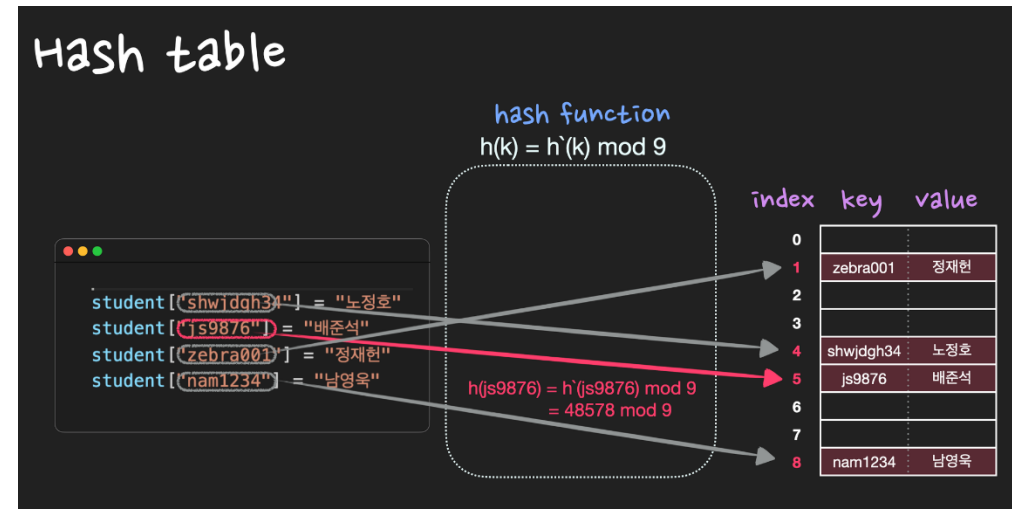
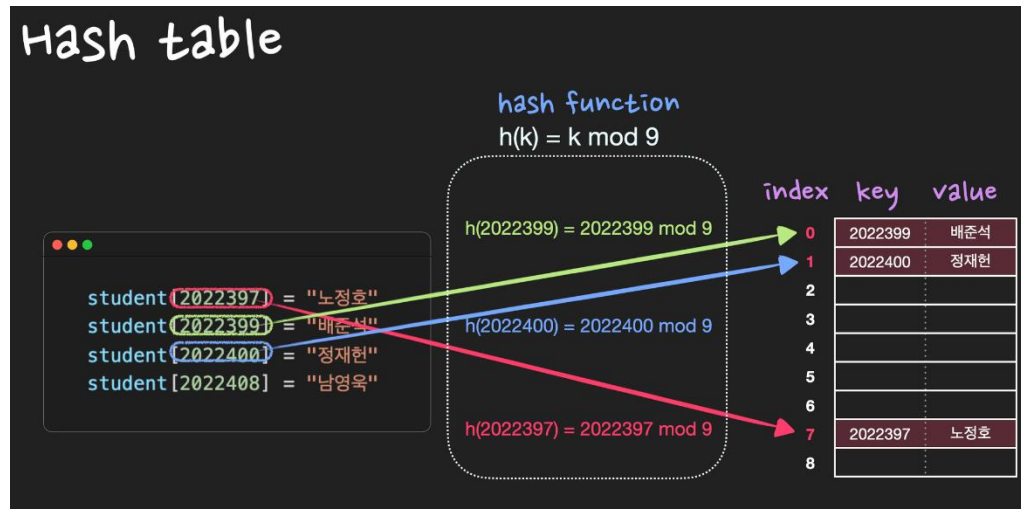
0	/
1	/
2	/
3	shwjdgh34 노정호
4	/
5	js9876 배준석
6	zebra001 정재현
7	nam1234 남영욱
8	/

HashTable

- 이와 같은 이유로, 직접 주소화 방법은 (key, value) 데이터 쌍을 저장하기 위한 방법으로 잘 맞지 않다. 그 대안으로 Hash table을 이용할 것이다. Hash table은 hash function h 를 활용해서 (key, value)를 저장한다. key 값을 k 라고 했을 때, $h(k)$ 함수 값에 해당하는 index에 (key, value) 데이터 쌍을 저장한다.

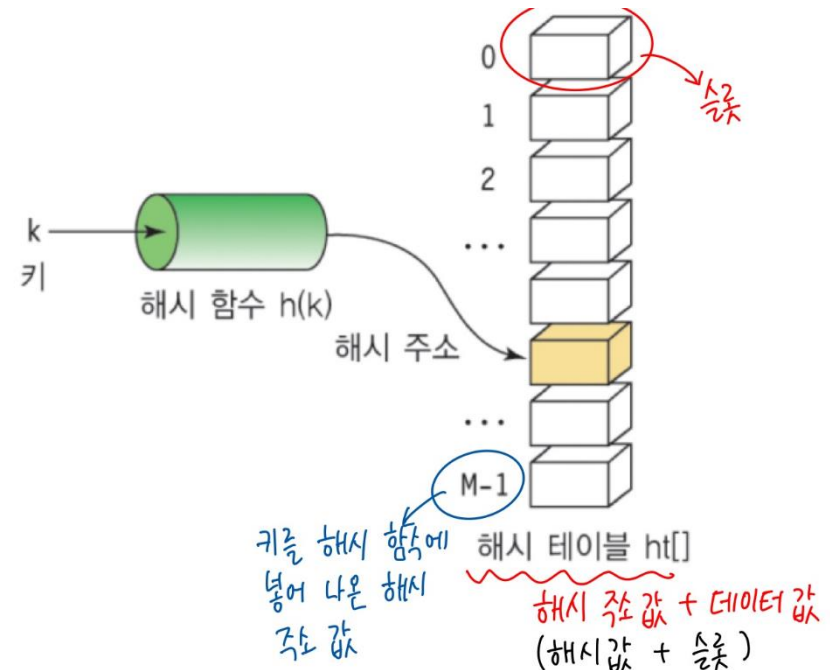
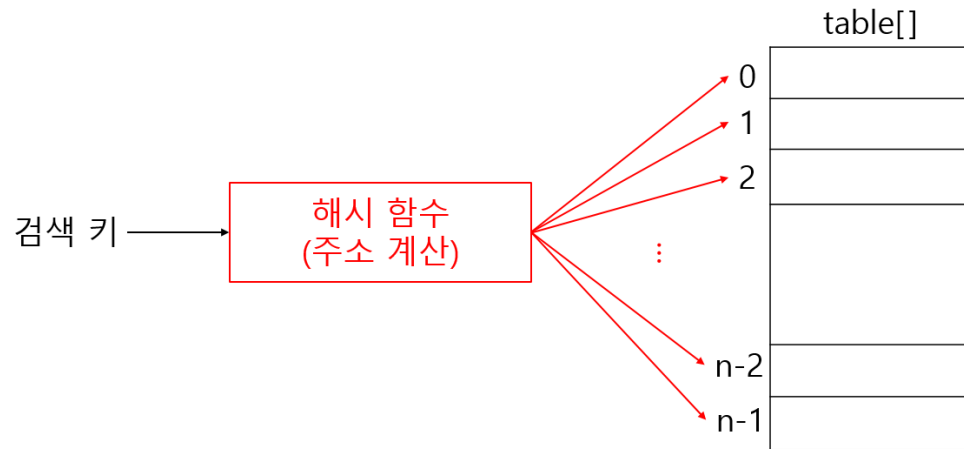
" $h(k)$ 는 키 k 의 해시값이다"라고 표현한다.

- 모든 데이터에 key 값은 무조건 존재해야 하며, 중복되는 key 값이 있어서는 안된다.
- (key, value) 데이터를 저장할 수 있는 각각의 공간을 slot또는 bucket이라고 한다.



Hash function

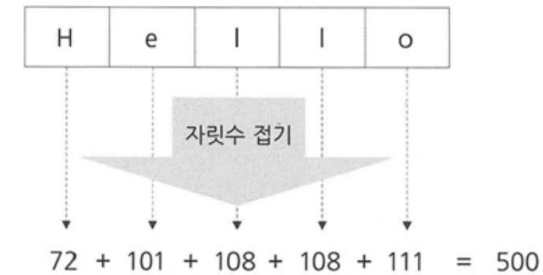
- 해시 함수(Hash Function)란, 주어진 데이터(Key)를 고유한 숫자 값인 해시 값(Hash Value)으로 표현해주는 함수이다.
- 키(Key)란, 해시 함수의 입력 부분이며, 입력 데이터 자체이거나 입력 데이터를 구분하는 값을 의미하며, 해시 값(hash value)이란, 해시 함수의 출력 부분이며, 해시 테이블(hash table)의 인덱스가 된다. 이러한 해시 함수를 통해 데이터를 고유한 숫자 값으로 변환하고, 해시 테이블에 접근하여 데이터를 조회하거나, 추출하게 된다.



Hash function

- 임의의 길이를 갖는 데이터를 고정된 길이의 데이터로 매핑하는 함수이다.
- 매핑 전 원래 데이터의 값을 키(key), 매핑 후 데이터의 값을 해시값(Hash Value) 또는 해시코드 라고 하며, 키(key)와 값(value)으로 매핑되는 과정 자체를 해싱(Hashing) 이라고 한다.
- 대표적인 해시 함수들
 - Division Method (나눗셈 법)
 - Digit Folding (자리수 접기)
 - Multiplication Method (곱셈 법)
 - Universal Hashing

$$h(k) = k \bmod N$$



$$h(k) = \lfloor N(kA \bmod 1) \rfloor$$

해시 함수와 충돌 처리

Hash function

- 해시 함수는 임의의 길이를 갖는 메시지를 입력받아서 고정된 길이의 해시값을 출력하는 함수다.
- 어떤 입력 값에도 항상 고정된 길이(해시 함수에 따라 비슷한 길이까지 포함)의 해시값을 출력한다.
- 해시 함수를 통해 입력값은 완전히 새로운 모습의 데이터로 만들어지기 때문에 암호화 영역에서 아주 주요하게 사용되고 있다.
- SHA(Secure Hash Algorithm)알고리즘이 그 대표적인 예시다.
- 또한 입력 값의 일부가 변경되면 전혀 다른 값을 출력하는데 이것을 가리켜 '눈사태 효과'라고 한다.
- 이 눈사태 효과로 결과값으로는 입력값을 유추할 수가 없다. 역추적이 안된다는 말은 '단방향'으로만 되어 있다는 의미를 내포한다.
- 하지만 이 해시 함수에도 단점이 존재하는데, 해시 함수는 입력값의 길이가 어쨌든 고정된 길이의 값을 출력하기 때문에 입력값이 다르더라도 같은 결과값이 나오는 경우가 있다.
- 이것을 '해시 충돌 hash collision'이라고 표현하며, 충돌이 적은 해시 함수가 좋은 해시함수다.



Load Factor(사용률)

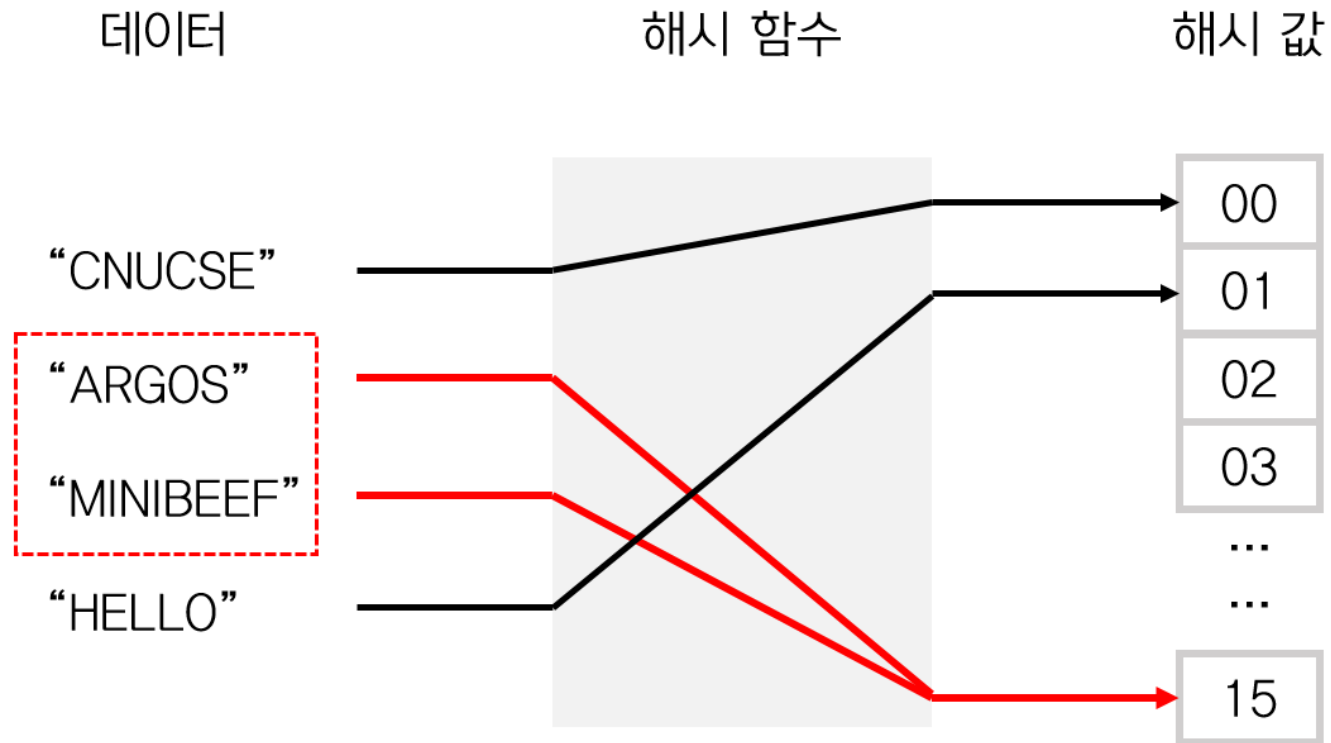
- 해시 테이블에서 사용률이란 해시 테이블의 공간을 얼마나 사용하고 있는지 수식으로 나타낸 것으로 다음과 같이 계산한다.

$$\frac{\text{해시 테이블에 저장된 항목 수}}{\text{해시 테이블 공간 수}}$$

- 만약에 배열에서 5개의 방이 있는 데 그 중 2개의 공간에 데이터가 저장되어 있다면 $2/5 = 0.4$ 가 사용률이 된다. 만약에 저장할 데이터가 100개라면 해시 테이블에는 최소 100개의 공간이 있어야 한다. 만약 100개의 데이터에 100개의 공간이 있다면 해시 테이블의 사용률은 1이 된다.
- 만약 100개의 데이터가 있는데 공간이 50개밖에 없다면 사용률은 2가 되고 절반의 데이터는 공간이 없게 된다. 즉, 사용률이 1보다 크다는 것은 해시 테이블의 공간이 부족하다는 뜻이다.
- 대개의 경우 사용률이 0.7 이상으로 커지면 해시 테이블을 리사이징하는데 대략 2배 정도의 크기로 배열을 만든다. 리사이징을 하여 배열을 크게 만들고 나면 해시 함수를 사용하여 기존 테이블에 있던 데이터를 새로운 해시 테이블에 다시 저장해야한다. 이 과정은 시간이 매우 오래 걸리기 때문에 리사이징을 자주 하는 것은 좋지 않다.

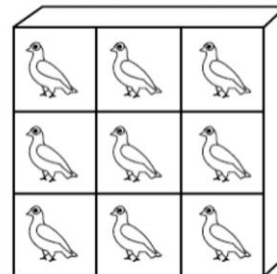
해시 충돌

- 다음 그림을 보자.
- ARGOS와 MINIBEEF가 같은 해시 값을 가지며 충돌이 발생했다.



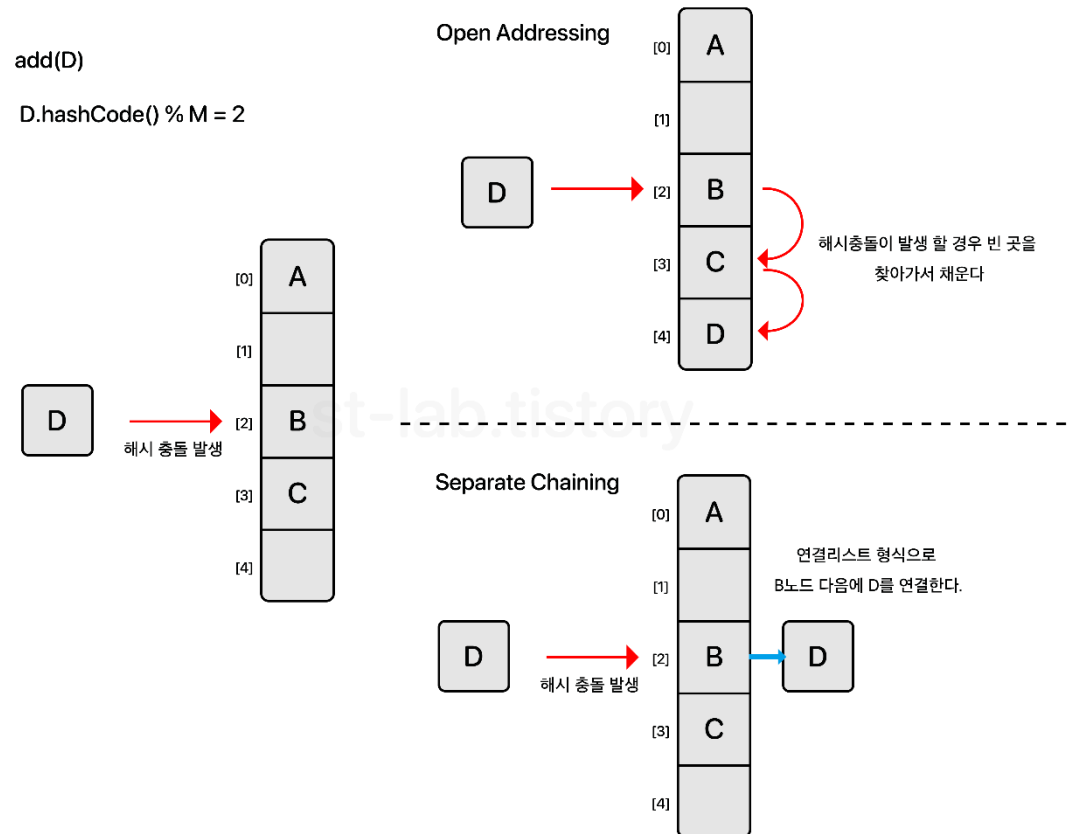
해시 충돌

- 해시 충돌을 설명하기 위해서는 적재율(load factor)이라는 개념이 필요하다.
- 적재율이란 해시 테이블의 크기에 대한 키의 개수의 비율을 뜻한다. 즉 키의 개수를 K , 해시 테이블의 크기를 N 이라고 했을 때 적재율은 K/N 이다.
- 만약 충돌이 발생하지 않을 경우 해시 테이블의 탐색, 삽입, 삭제 연산은 모두 $O(1)$ 에 실행되지만, 충돌이 발생할 경우에는 탐색과 삭제 연산이 $O(K)$ 만큼 걸리게 된다.
- 해시 충돌이 1도 없는 해시 함수를 만드는 것은 불가능하다. 따라서 해시 충돌에 대해 안전하다는 해시 함수는 '충돌을 찾는 게 거~의 희박하다'라는 뜻이다.
- 이 해시 충돌이 발생하는 근본적인 원인은 비둘기집 원리다.
- 즉 해시 함수가 무한한 가짓수의 입력값을 받아 유한한 가짓수의 출력값을 생성하는 경우에는 비둘기집 원리에 의해 해시 충돌은 항상 존재한다. 따라서 해시 테이블의 충돌을 완화하는 방향으로 문제를 보완해야 한다.



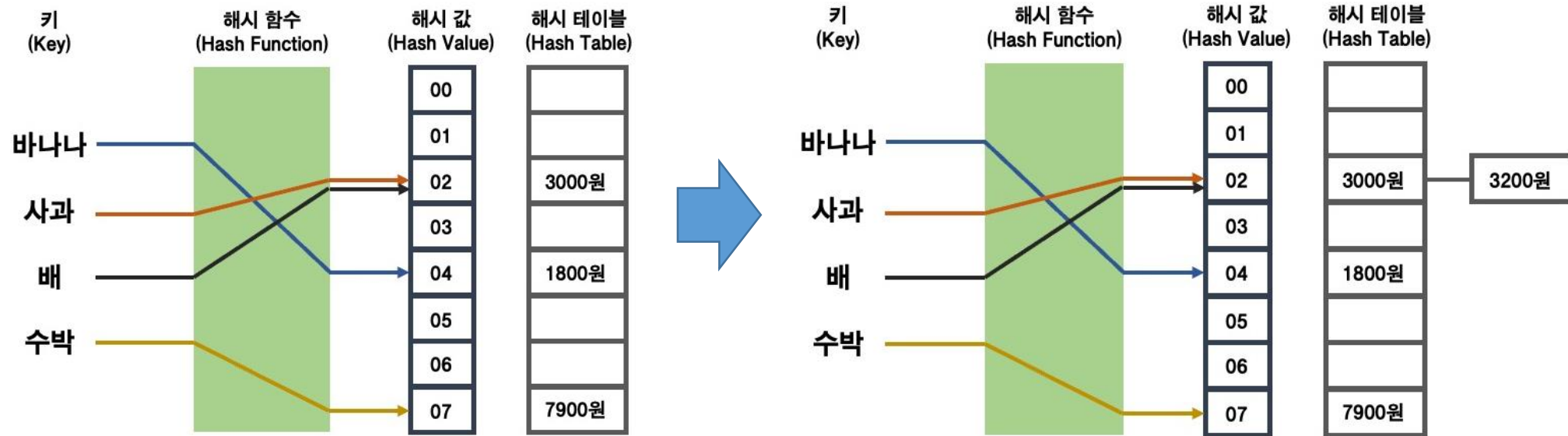
해시 충돌

- 해시 충돌을 완화하는 데는 다양한 방법이 있다.
- 개방 주소법(open addressing)으로 해시 테이블 크기는 고정하면서 저장할 위치를 잘 찾거나, 분리 연결법 (seperate chaining)으로 해시 테이블의 크기를 유연하게 만드는 방법이 대표적이다.



Chaining 기법

- Chaining 기법은 Open Hashing 기법 중 하나로, 해쉬 테이블 저장공간 이외의 공간을 활용하는 기법이다.
- 즉, 충돌이 일어나게 되면 해당 인덱스가 가리키는 해쉬 테이블 공간 뒤로 연결 리스트를 사용하여 추가적으로 연결시킨 다음 저장한다.



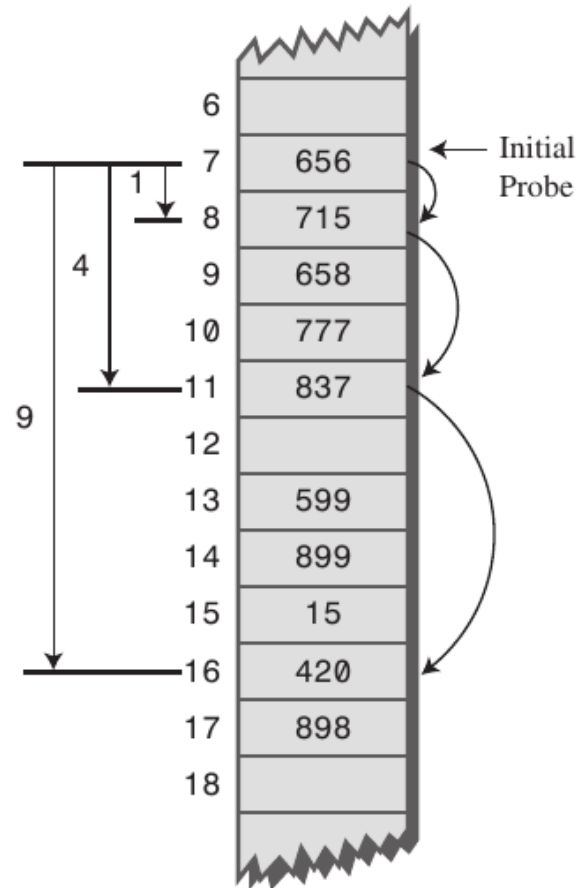
개방 주소법(Open Addressing)

- Linear Probing 기법
- Linear Probing 기법은 Close Hashing 기법 중 하나로, 해시 테이블 저장공간 안에서 충돌 문제를 해결하는 기법이다.
- 충돌이 일어나게 되면, 해당 해시 값의 다음 값부터 순회를 시작하여 처음으로 나오는 해시 테이블의 빈 공간에 저장하는 기법이다.



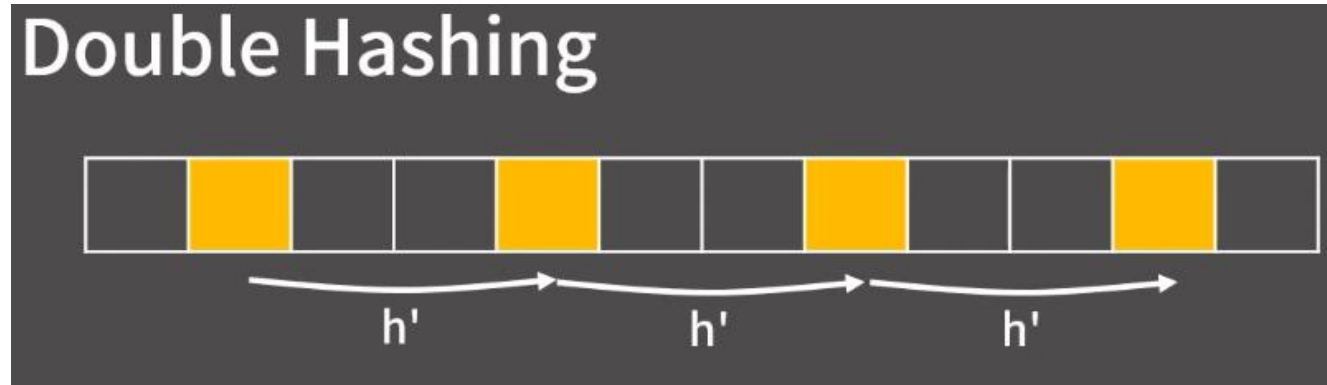
개방 주소법(Open Addressing)

- 제곱 탐사(Quadratic Probing)
- 제곱 탐사(Quadratic Probing)은 해시의 저장 순서 폭을 제곱으로 저장하는 방식이다.
- 예를 들어 처음 충돌이 발생하면 1만큼 건너뛰고, 다음 충돌이 발생하면 2^2 , 3^2 만큼... 으로 탐사를 한다.



개방 주소법(Open Addressing)

- 이중 해싱(Double hashing)
- 충돌이 생기면 2차 해시 함수로 다시 해싱을 해서 새로운 주소를 할당하는 방법이다



HashTable

- 연관 배열(Associative Array), 맵(Map), 사전(Dicitionary) 등으로도 불리기도 한다.
- 대부분 언어에서 라이브러리로 이 자료구조를 지원하기 때문에 굳이 직접 구현할 필요는 없다.
 - Dictionary (Python)
 - Hash - (Ruby, Perl)
 - Map - (Java, Go, Haskell 등)
 - Unordered_map, Map - (C++)
- 하지만 이를 본질적으로 이해하기 위해선 직접 구현할 필요가 있다.
- 그리 복잡하지는 않으니 꼭 한번 구현해보자.
- 구현할 때에는 배열과 리스트로 구현한다.

구현해야 하는 이유

해시테이블 사용법 - dictionary

- 파이썬의 딕셔너리는 hash table로 구현되어 있다.

```
hashtable = {}
```

- hash table은 앞서 key-value의 쌍으로 이루어졌다고 했다. 파이썬에서는 key를 리스트의 index처럼 생각하면 편하다.
- 예를 들어 리스트에서 value를 access하거나 update할 때, index를 사용했다.

```
a = [1, 3, 5, 7]
```

```
a[3] # 7
```

```
a[3] = 0
```

- dictionary도 마찬가지다. 아래와 같이 key-value가 있을 때, key를 index처럼 생각해주는 것이다.

key: 학번, value: 이름

(2022390, 노정호)

(2022392, 배준석)

(2022393, 정재헌)

(2022401, 남영욱)

```
student_info = {}
```

```
student_info[2022390] = "노정호"
```

```
student_info[2022392] = "배준석"
```

```
student_info[2022393] = "정재헌"
```

```
student_info[2022401] = "남영욱"
```


해시테이블 사용법 - dictionary

- in 연산자
- dictionary는 in 연산자와 결합했을 때 강력한 힘을 발휘한다. in은 dictionary에서 key가 존재하는지 확인해준다.
- 만약 key가 존재하면 True를 반환하고 존재하지 않으면 False를 반환한다.

```
if 2022390 in student_info:
```

```
    print("학생이 존재합니다")
```

```
else:
```

```
    print("학생이 존재하지 않습니다")
```

- 그리고 이 in 연산자의 시간복잡도는 $O(1)$ 으로 매우 효율적이다.
- list도 in 연산자를 사용하여, value가 있는지 없는지 확인할 수 있지만 $O(n)$ 의 시간복잡도를 갖는다.

해시테이블 사용법 - dictionary

- Dictionary 관련 함수

1. dictionary.item()

key와 value 모두 접근할 때 사용한다.

```
for student_id, name in student_info.items():  
    print(student_id, name)
```

2. dictionary.keys()

dictionary의 key들을 접근할 때 사용한다.

```
for student_id in student_info.keys():  
    print(student_id)
```

3. dictionary.values()

dictionary의 value들을 접근할 때 사용한다.

```
for name in student_info.values():  
    print(name)
```

해시테이블 사용법 - dictionary

4. dictionary.get()

key에 해당하는 value을 가져올 때 사용한다.

```
print(student_info.get(2022390))
```

만약 다음과 같이 없는 값을 가져오면 어떻게 될까요?

```
print(student_info.get(1111))
```

이렇게 없는 값을 가져올 경우, default를 지정할 수도 있다.

```
print(student_info.get(1111, "김기영"))
```

조금 더 응용하면, 매우 편리하게 사용할 수 있다.

```
if 3 not in a:
```

```
    a[3] = 1
```

```
else:
```

```
    a[3] += 1
```

위와 같은 코드를 단 1줄만으로도 작성할 수 있다.

```
a[3] = 1 + a.get(3, 0)
```

해시테이블 사용법 - dictionary

- 그럼 딕셔너리가 만능일까요?
- 지금까지만 보면 딕셔너리는 list의 완벽한 상위호환 같다.
- 그러나 list도 list의 장점이 있다.
- 그것은 순서가 있다는 것이다. 리스트와 다르게 dictionary는 정렬할 수 없다.

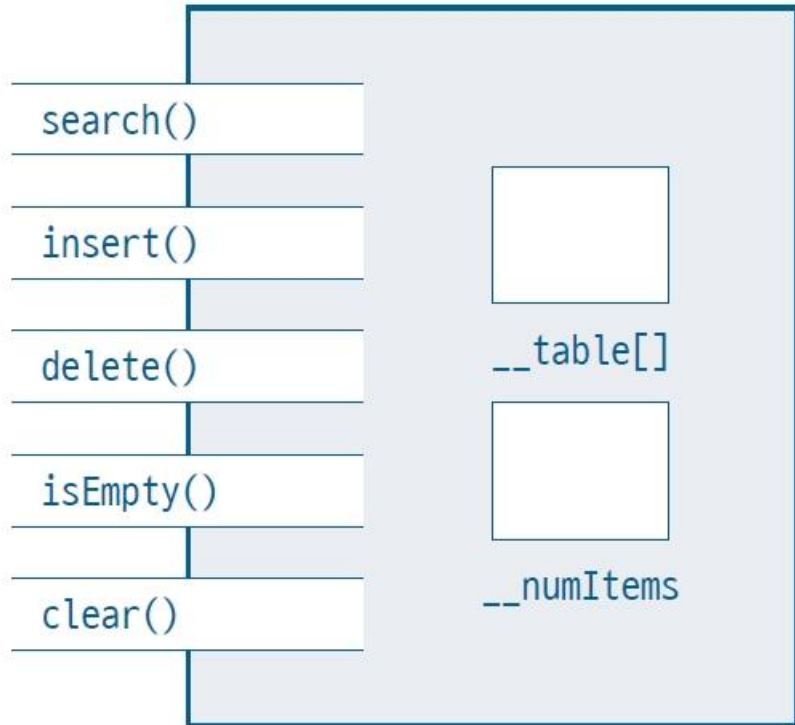
```
sorted_dict = sorted(my_dict.items())
```

- 다음과 같이 정렬한다 하더라도, 결국은 list로 반환된다.
- 그렇기에 데이터의 순서가 중요할 때는 list를 사용하는 것이 바람직하다.

■ 점프 투 파이썬 / 02장 파이썬 프로그래밍의 기초, 자료형 / 02-5 딕셔너리 자료형

02-5 딕셔너리 자료형

해시 테이블 객체 구조



`__table[]` ◀ 해시 테이블로 사용하는 배열

`__numItems` ◀ 해시 테이블의 원소 개수

`search(x)` ◀ 해시 테이블에서 원소 `x`를 검색한다.

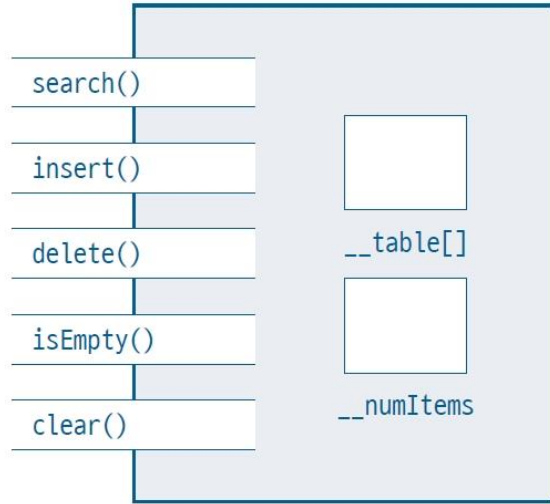
`insert(x)` ◀ 해시 테이블에 원소 `x`를 삽입한다.

`delete(x)` ◀ 해시 테이블에서 원소 `x`를 삭제한다.

`isEmpty()` ◀ 해시 테이블이 빈 상태인지 알려준다.

`clear()` ◀ 해시 테이블을 깨끗이 청소한다.

파이썬 구현



해시 테이블 객체 구조

코드 12-1 chainedHashTable.py

```

1 from DS.list.CircularLinkedList import *
2 from DS.list.listNode import *
3
4 class ChainedHashTable:
5     def __init__(self, n):
6         self.__table = [CircularLinkedList() for i in range(n)]
7         self.__numItems = 0;
8
9     def __hash(self, x:int): # 편의상 int 타입으로 제한
10        return x % len(self.__table)
11

```

파이썬 구현

```
12 # [알고리즘 12-1] 구현: 검색, 삽입, 삭제
13 def insert(self, x:int):
14     slot = self.__hash(x)
15     self.__table[slot].insert(0, x)
16     self.__numItems += 1
17
18 def search(self, x:int) -> ListNode:
19     slot = self.__hash(x)
20     if self.__table[slot].isEmpty():
21         return None # null list
22     else:
23         head = prev = self.__table[slot].getNode(-1) # 더미 헤드
24         curr = prev.next # 0번 노드
25         while curr != head:
26             if curr.item == x:
27                 return curr
28             else:
29                 prev = curr; curr = curr.next
30         return None
```

파이썬 구현

알고리즘 12-2 개방 주소 방법

```

◀ table[]: 해시 테이블
insert(x):
    i ← 0
    repeat
        j ← hi(x)
        if (table[j] = null or table[slot] = DELETE)
            table[j] ← x
            return j
        else i++
    until (i = m)
    error "테이블 오버플로우"
    
```

알고리즘 12-2 개방 주소 방법

```

search(x):
    i ← 0
    repeat
        j ← hi(x)
        if (table[j] = x) return j
        else i++
    until (table[j] = null or i = m)
    return NOT_FOUND ◀ search failed!
    
```


파이썬 구현

알고리즘 12-2 개방 주소 방법

```
delete(x):  
    i ← 0  
    repeat  
        j ← hi(x)  
        if (table[j] = x)  
            ❶ table[j] ← DELETED;  
            break  
        else i++  
    until (table[j] = null or i = m)
```

파이썬 구현

코드 12-3 hashOpenAddressed.py

```
1 class HashOpenAddressed:
2     def __init__(self, n:int):
3         self.__table = [None for i in range(n)]
4         self.__numItems = 0
5         self.__DELETED = -54321
6
7     def __hash(self, i:int, x):
8         return (x + i) % len(self.__table)
9
10    def insert(self, x):
11        if self.__numItems == len(self.__table):
12            """ 에러 처리 """
13        else:
14            for i in range(len(self.__table)):
15                slot = self.__hash(i, x)
16                if self.__table[slot] == None or self.__table[slot] == self.__DELETED:
17                    self.__table[slot] = x
18                    self.__numItems += 1
19                    break
20
```

Hash table

파이썬 구현

```
21 NOT_FOUND = -12345 # constant
22 def search(self, x) -> int:
23     for i in range(len(self.__table)):
24         slot = self.__hash(i, x)
25         if self.__table[slot] == None:
26             return HashOpenAddressed.NOT_FOUND
27         if self.__table[slot] == x:
28             return slot # Found at self.__table[slot]
29     return self.__NOT_FOUND
30
31 def delete(self, x):
32     for i in range(len(self.__table)):
33         slot = self.__hash(i, x)
34         if self.__table[slot] == None:
35             break
36         if self.__table[slot] == x:
37             self.__table[slot] = self.__DELETED;
38             self.__numItems -= 1
39             break
```

파이썬 구현 예제

```
import hashlib

class HashTable:
    def __init__(self):
        # 해시 테이블 0으로 초기화
        self.hash_table = [0 for i in range(8)]

    # 문자열을 정수로 바꿔주어 key가 되게끔 함
    def get_key(self, data):
        # SHA-256 알고리즘 사용
        hash_convert = hashlib.sha256()
        hash_convert.update(data.encode())
        # 16진수로 return
        return int(hash_convert.hexdigest(), 16)

    # 간단하게 나머지를 이용한 해시 함수 구현
    def hash_function(self, key):
        return key % 8

    # 데이터를 해시 테이블에 저장하는 함수
    def save(self, data, value):
        # 키 생성
        key = self.get_key(data)
        # 생성된 키를 해시 함수에 투입
        hash_value = self.hash_function(key)
```

파이썬 구현 예제

```
# 해시 테이블의 해당 해시 값 부분이 비어있지 않으면(초기화한 값 0이 아니면)
if self.hash_table[hash_value] != 0:
    for index in range(len(self.hash_table[hash_value])):
        # 한 hash address 안에 리스트가 있고, 그 리스트 안에는
        # key와 value값으로 이루어진 리스트가 또 존재함
        # ([key, value]를 요소로 가지는 연결 리스트)

        # 해시 값이 같을 때, chaining 기법으로 연결 된 연결 리스트를 순회하다가
        # key값도 같은 요소가 있을 시 해당 key의 value값 update
        if self.hash_table[hash_value][index][0] == key:
            self.hash_table[hash_value][index][1] = value
            return

    # 해시 값이 같을 때, key값이 같은 요소가 없었으면
    # 해당 해시 값을 인덱스로 하는 해시 테이블에 [key, value]를 요소로 하는 연결 리스트 추가
    self.hash_table[hash_value].append([key, value])

# 해시 테이블의 해당 해시 값 부분이 비어있으면(초기화한 값 0이면)
else:
    # [key, value]를 요소로 하는 연결 리스트 추가
    self.hash_table[hash_value] = [[key, value]]
```

파이썬 구현 예제

```
# 해시 테이블의 데이터 조회하는 함수
def read(self, data):
    # 키 생성
    key = self.get_key(data)
    # 생성된 키를 해시 함수에 투입
    hash_value = self.hash_function(key)

    # 해시 테이블의 해당 해시 값 부분이 비어있지 않으면(초기화한 값 0이 아니면)
    if self.hash_table[hash_value] != 0:
        for index in range(len(self.hash_table[hash_value])):
            # 해시 테이블의 해당 해시 값 부분에 key값도 같은 요소가 있으면
            # 해당 key값의 value값 return
            if self.hash_table[hash_value][index][0] == key:
                return self.hash_table[hash_value][index][1]
        return None

    # 해당 해시 값 부분에 데이터가 들어가 있지 않으면
    # 데이터가 없는 것이기 때문에 none값 return
    else:
        return None
```

```
hash_table = HashTable()
hash_table.save("배", "3200")
hash_table.save("사과", "3000")
hash_table.save("바나나", "1800")
hash_table.save("수박", "7900")
print(hash_table.read("배"))
print(hash_table.read("사과"))
```


Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 강민, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- 자료구조와 알고리즘 with 파이썬, 최영규, 생능북스
- <https://wikidocs.net/195736>
- <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>
- <https://calcflx.com/%ED%85%8D%EC%8A%A4%ED%8A%B8-%EC%95%94%ED%98%B8%ED%99%94-%EC%82%AC%EC%9D%B4%ED%8A%B8/>
- <https://wikidocs.net/16>